ROSMASTER Crash Course

What is ROS 2?

Week 1: ROS 2, Gazebo, and a robot that avoids walls

Phan Phearamony
KAMAL Laboratory, Gunma University

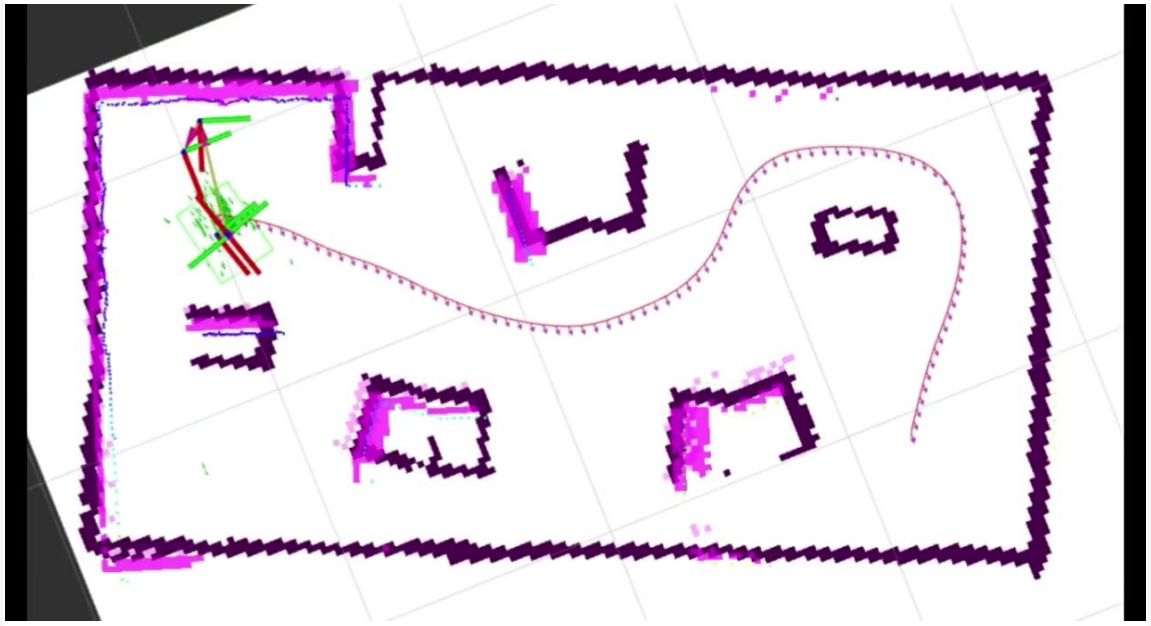


Yahboom ROSMASTER A1

By Week 4 you will understand how this works



Real ROSMASTER A1



What the robot sees (RViz2)

Known map — Surprise obstacles — Unknown room

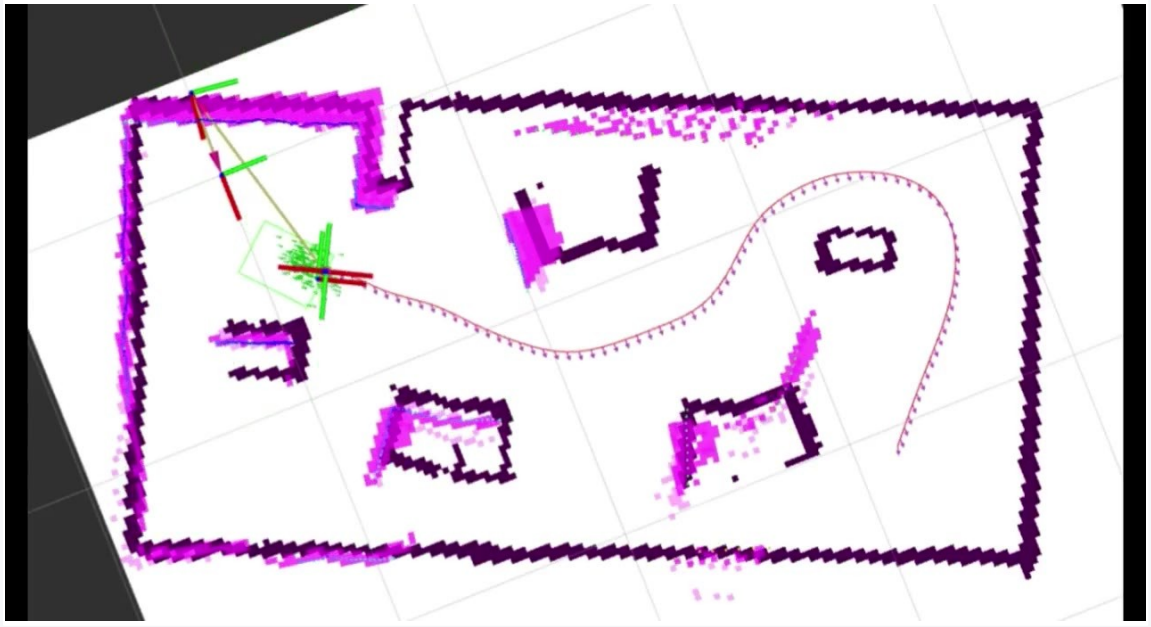
Scenario 1: Known map

The robot already has a map of the room. It plans a path and drives to the goal.

By Week 4 you will understand how this works



Real ROSMASTER A1



What the robot sees (RViz2)

Known map — **Surprise obstacles** — Unknown room

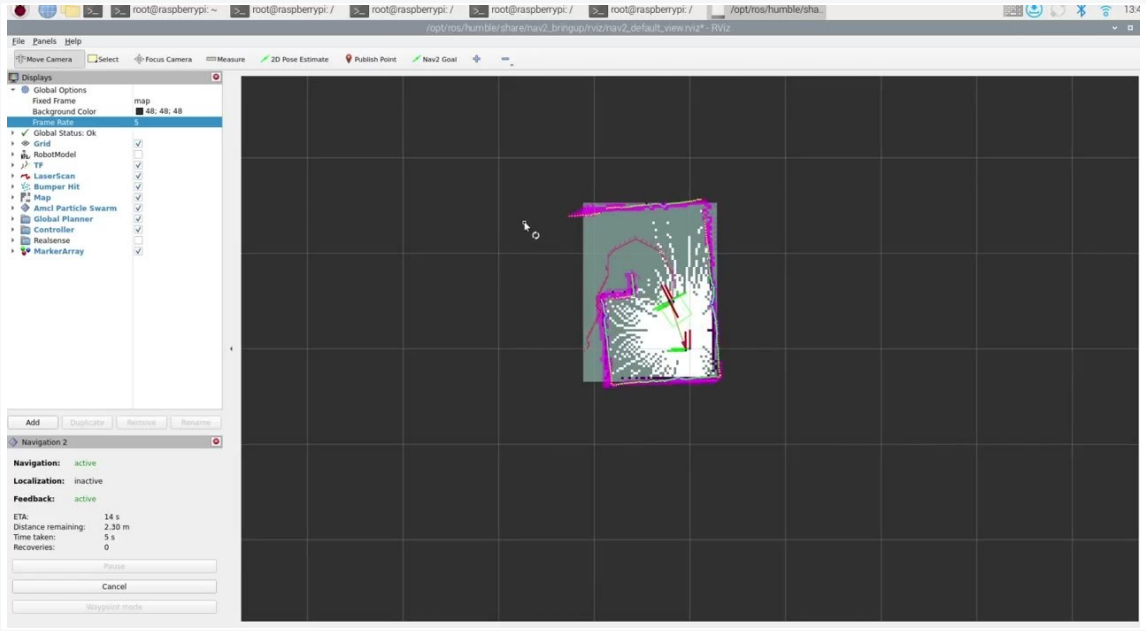
Scenario 2: Surprise obstacles

Same map, but we add boxes the map does not know about. The robot sees them with LiDAR and goes around.

By Week 4 you will understand how this works



Real ROSMASTER A1



What the robot sees (RViz2)

Known map — Surprise obstacles — **Unknown room**

Scenario 3: Unknown room

No map at all. The robot builds the map while it drives, and still finds the goal.

Course plan

WEEK 1

ROS 2 Introduction

WEEK 2

Reading the Code

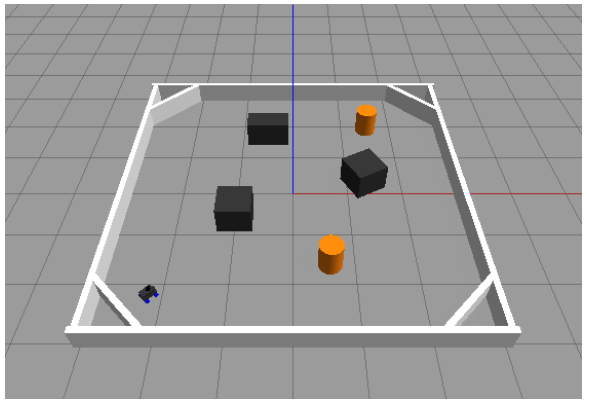
WEEK 3

Control and Sensors

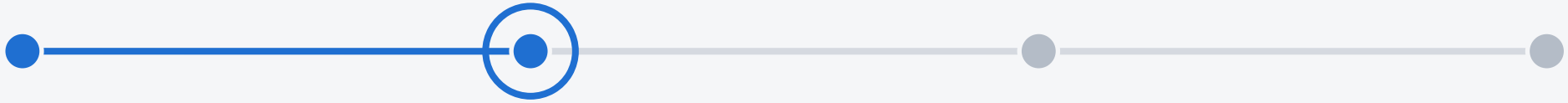
WEEK 4

ROSMASTER Functions

- What ROS 2 is and why robots use it
- Nodes, topics and messages
- ROS vs ROS 2, and the versions
- Gazebo, the robot simulator
- Hands-on: a simulated A1 avoids obstacles with LiDAR



Course plan



WEEK 1
ROS 2 Introduction

WEEK 2
Reading the Code

WEEK 3
Control and Sensors

WEEK 4
ROSMASTER Functions

- Linux: files, folders and the terminal
- Bash commands you use every day: cd, ls, nano, source, sudo apt
- Python basics: variables, if, functions, classes
- Read the Week 1 avoidance node line by line

```
def scan_callback(...):
    front = ...
    if front < safe:
        turn()
```

Course plan

WEEK 1
ROS 2 Introduction

WEEK 2
Reading the Code

WEEK 3
Control and Sensors

WEEK 4
ROSMASTER Functions

- How the A1 moves: /cmd_vel, Ackermann steering, odometry
- Its sensors: LiDAR, camera, IMU, wheel encoders
- Example: LiDAR obstacle avoidance on the real A1, like Week 1
- Homework: use the A1's built-in AI (YOLO) to recognize objects



Course plan

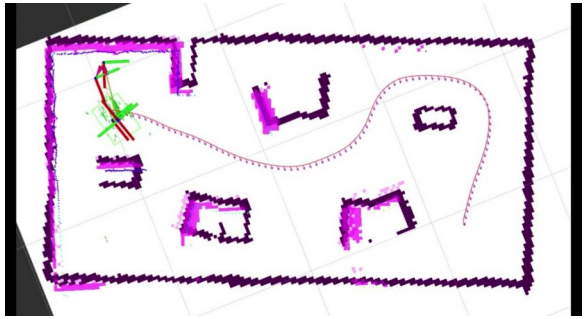
WEEK 1
ROS 2 Introduction

WEEK 2
Reading the Code

WEEK 3
Control and Sensors

WEEK 4
ROSMASTER Functions

- Tour the A1's built-in functions from the Yahboom docs
- Mapping (SLAM) and Nav2 navigation
- LiDAR follow and patrol, line following, color tracking
- AI functions: object recognition and voice or text commands



Part 1

What is ROS 2?

The main ideas, roughly. We go through the real code in Week 2.



Sense, think, act



Sense

LiDAR, camera, IMU



Think

Raspberry Pi or Jetson



Act

Motors and steering servo

Any robot, from a vacuum cleaner to a self-driving car, has these three parts.

Nodes

In ROS 2 each part runs as its own small program, called a node. ROS 2 itself is not an operating system. It is a set of tools on top of Linux that lets these programs send data to each other.



Because they are separate, you can replace one node (say, a new LiDAR driver) and leave the others alone.

Topics and messages

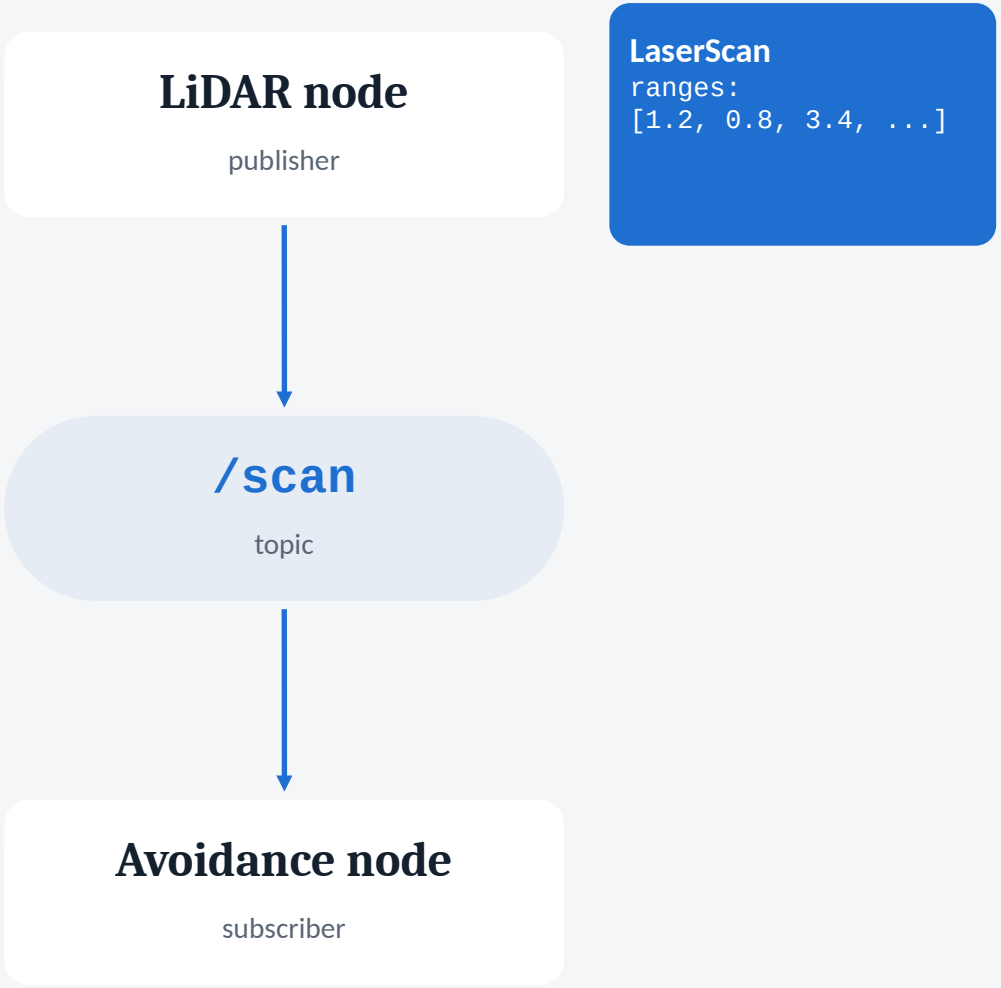
Node — A program that does one job.

Topic — A named channel, like /scan. Anyone can listen.

Message — The data sent on a topic, with a fixed type.

Publisher — A node that sends messages to a topic.

Subscriber — A node that receives messages from a topic.



Topics and messages

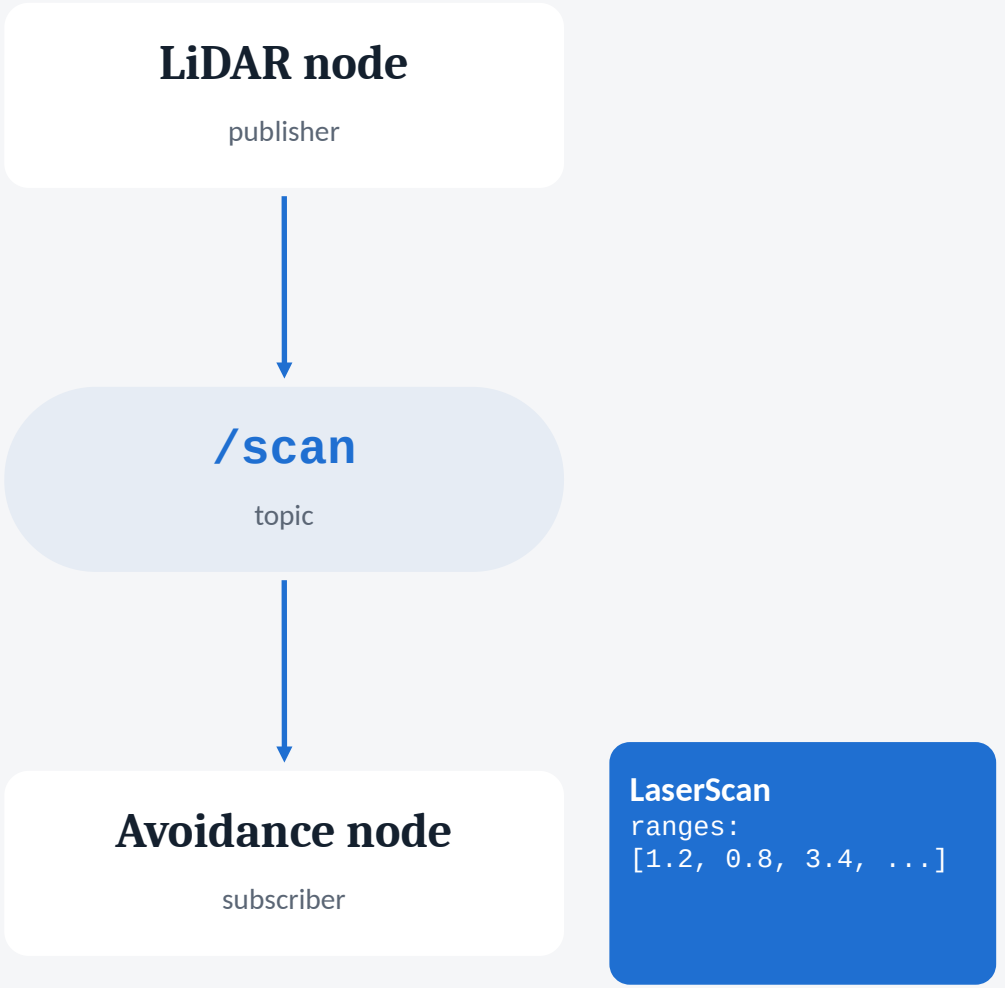
Node — A program that does one job.

Topic — A named channel, like /scan. Anyone can listen.

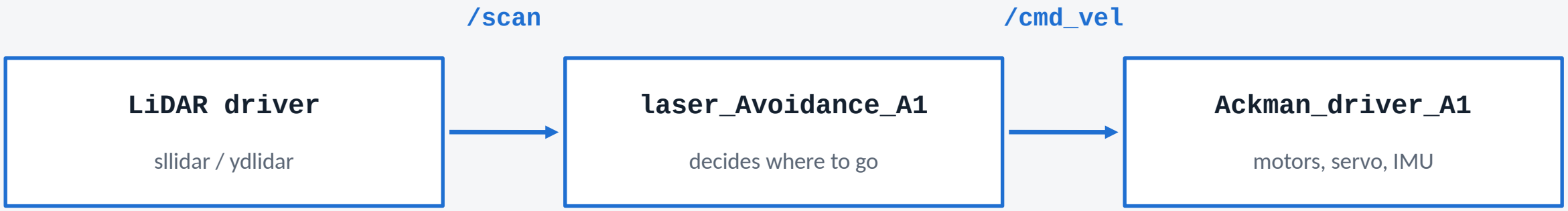
Message — The data sent on a topic, with a fixed type.

Publisher — A node that sends messages to a topic.

Subscriber — A node that receives messages from a topic.



The nodes on our A1





Yahboom's commands for these three (we use them in Week 3):

```
# start the chassis (motors + servo)
ros2 run yahboomcar_bringup Ackman_driver_A1
# start the LiDAR (C1 model)
ros2 launch sllidar_ros2 sllidar_c1_launch.py
# start obstacle avoidance
ros2 run yahboomcar_laser laser_Avoidance_A1
```

Common topics on the A1

Topic	Message type	What it carries
`/scan`	`sensor_msgs/LaserScan`	LiDAR distances, 360 degrees
`/cmd_vel`	`geometry_msgs/Twist`	Speed and turning command to the robot
`/odom`	`nav_msgs/Odometry`	Where the robot thinks it is
`/imu`	`sensor_msgs/Imu`	Rotation and acceleration
`/map`	`nav_msgs/OccupancyGrid`	2D map: free, wall, or unknown
`/camera/image_raw`	`sensor_msgs/Image`	Camera picture

Today we only use /scan and /cmd_vel.

ROS 1 and ROS 2

ROS 1

- Started in 2007
- Needs one central master (roscore). If it dies, everything stops.
- Built for one robot on a good network
- Last version Noetic. End of life May 31, 2025.

ROS 2

- First release December 2017
- No master. Nodes find each other on the network by themselves (DDS).
- Built for many robots, Wi-Fi, and real products
- Actively developed. The A1 runs ROS 2.

ROS 2 versions



Lyrical Luth (newest)

Released May 22, 2026. Supported until May 2031.

A new version comes out every May. Even-year versions are LTS, which means 5 years of updates.

ROS 2 versions



Humble Hawksbill (what we use)

Ubuntu 22.04 · supported until May 2027

The A1 ships with Humble, and so does our lab setup. Each ROS 2 version pairs with one Ubuntu version, so install Ubuntu 22.04 with it.

The workspace

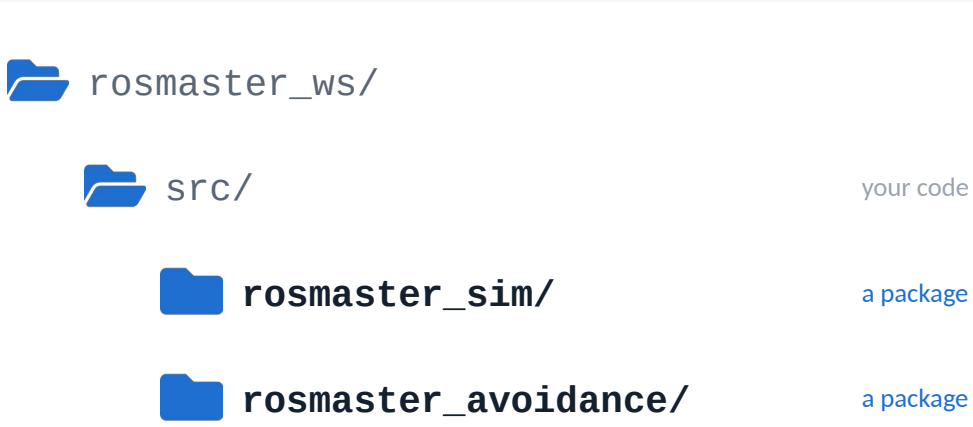


A workspace is the folder that holds one robot project.

Your code goes in src/. The workspace name can be anything.

```
# make both folders at once
mkdir -p ~/rosmaster_ws/src
```

Packages



A package is a folder for one part of the project. ROS 2 builds and runs code package by package.

Our course has two of them:

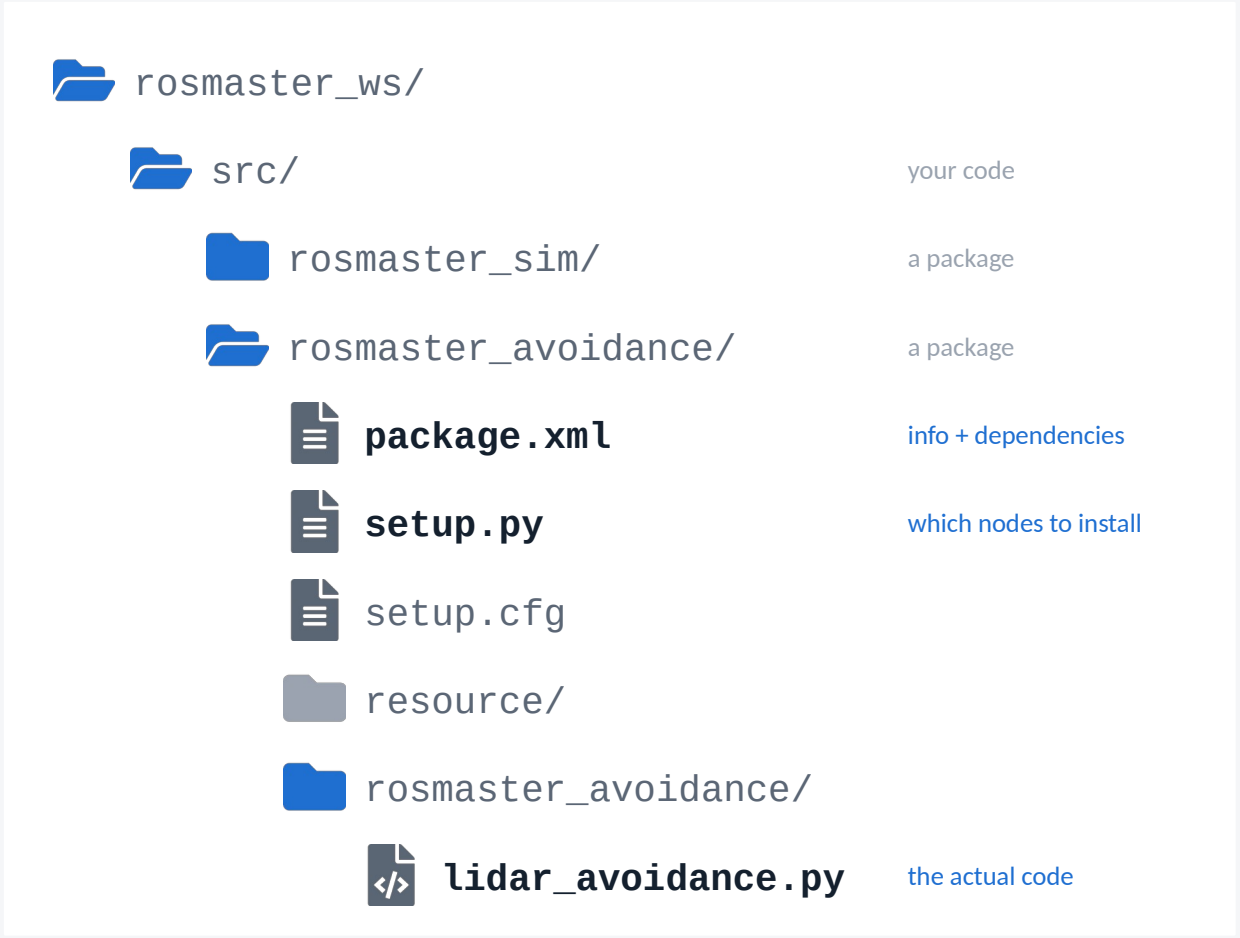
`rosmaster_sim`

The A1 robot model, the Gazebo world, and the launch file that starts them.

`rosmaster_avoidance`

Our Python node: reads /scan, publishes /cmd_vel.

Inside a package



`package.xml`

Name, version, and the packages it depends on (rclpy, sensor_msgs).

`setup.py`

Tells ROS which nodes this package has, so ros2 run can find them.

`lidar_avoidance.py`

The robot logic. We read this file in Week 2.

ros2 pkg create makes these files for you:

```
ros2 pkg create --build-type ament_python \
  rosmaster_avoidance
```

Build, source, run



colcon builds everything in src/ and creates build/, install/ and log/. You don't edit those.

1 **Build**

```
colcon build
```

2 **Source**

```
source install/setup.bash
```

3 **Run**

```
ros2 run / ros2 launch
```

"Package not found" usually means the terminal was not sourced.

Commands for today

```
ros2 run <package> <node>          # start one node
ros2 launch <package> <file>       # start many nodes at once

ros2 node list                     # which nodes are running?
ros2 topic list                    # which topics exist?
ros2 topic echo /scan              # print messages live
ros2 topic hz /scan                # how fast are they coming?

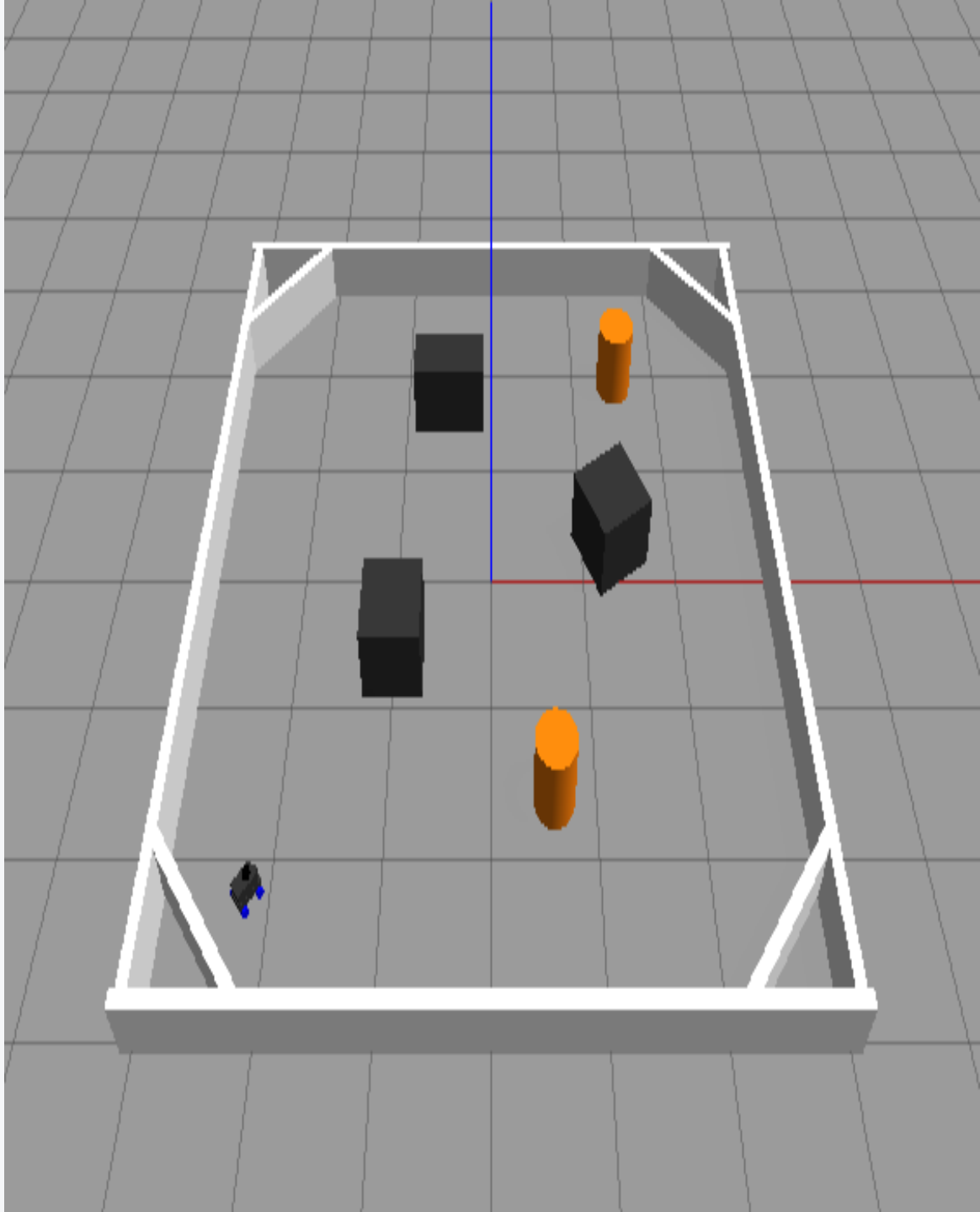
rqt_graph                          # draw the node graph
```

Ctrl+C stops whatever is running in a terminal.

Part 2

Gazebo

The simulator we use before moving to the real robot.



Gazebo

Physics

Gravity, friction and collisions, close to the real thing.

Simulated sensors

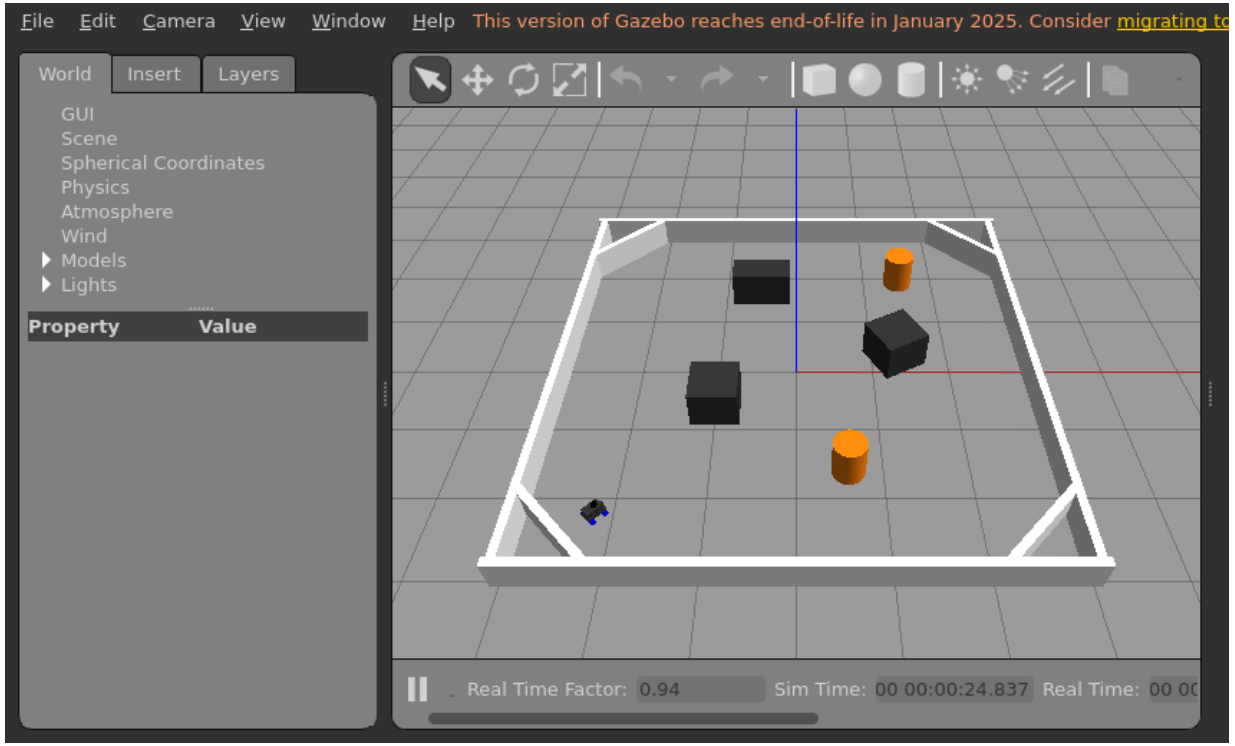
LiDAR, camera and IMU that give realistic data.

Same topics

Plugins publish /scan and read /cmd_vel, like the real robot does.

Nothing breaks

You can crash into walls as often as you like.

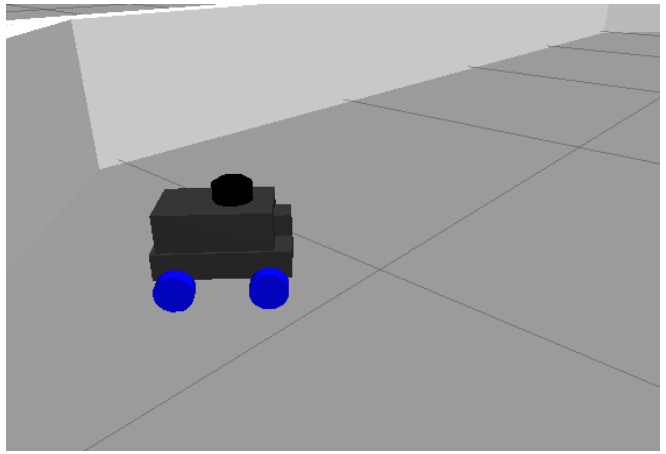


Our 6 x 6 m room and A1 model in Gazebo Classic 11, the version that comes with Humble. Classic reached end of life in January 2025. The newer Gazebo (Harmonic and later) is worth learning after this course.

Simulation and real robot

Gazebo

Week 1 and 2



`lidar_avoidance`

same code in both

`/scan in`
`/cmd_vel out`

Real ROSMASTER A1

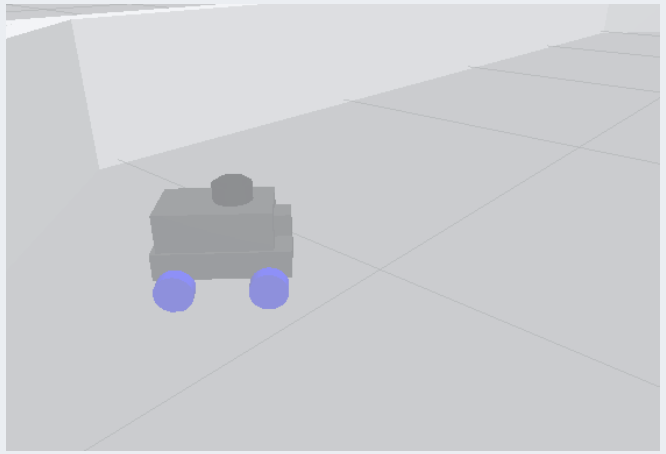
Week 3 and 4



Simulation and real robot

Gazebo

Week 1 and 2



`lidar_avoidance`

same code in both

`/scan in`
`/cmd_vel out`

Real ROSMASTER A1

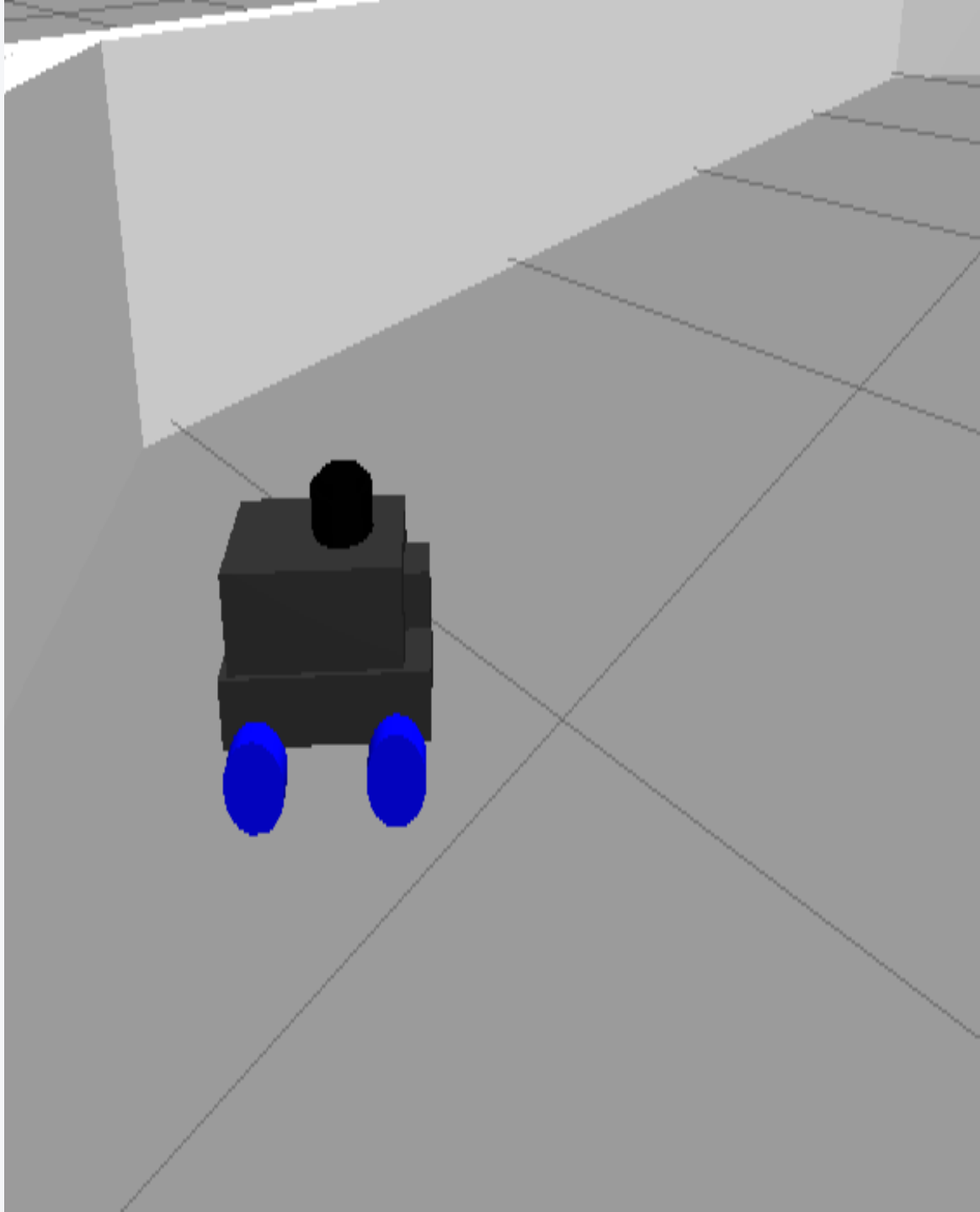
Week 3 and 4



Part 3

Hands-on

Install, run the simulation, and see how the avoidance node works.



Installing on Windows (WSL2)

1. Install Ubuntu (WSL2) — 2. Prepare Ubuntu — 3. Add the ROS 2 source — 4. Install ROS 2 + Gazebo — 5. Load ROS 2 + test — 6. Test Gazebo

Windows PowerShell (Run as administrator)

```
# Right click PowerShell > Run as administrator
wsl --install -d Ubuntu-22.04

# Restart the PC. Open "Ubuntu 22.04" from the Start menu.
# Make a username and password. The password is invisible while you type.

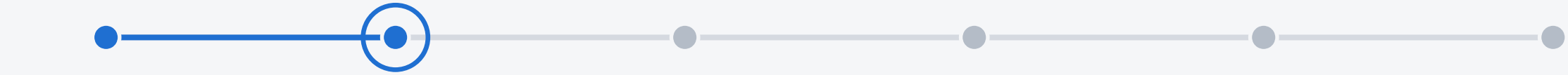
# Check, inside Ubuntu:
lsb_release -a          # should say Ubuntu 22.04
```

WSL2 runs a real Ubuntu 22.04 inside Windows. ROS 2 Humble needs Ubuntu 22.04.

Note

Needs Windows 11, or Windows 10 build 19044+ (press Win+R, type winver). Already have WSL? Run wsl --update first.

Installing on Windows (WSL2)



Ubuntu terminal

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y locales software-properties-common curl git

# ROS 2 needs a UTF-8 language setting
sudo locale-gen en_US en_US.UTF-8
sudo update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8
export LANG=en_US.UTF-8

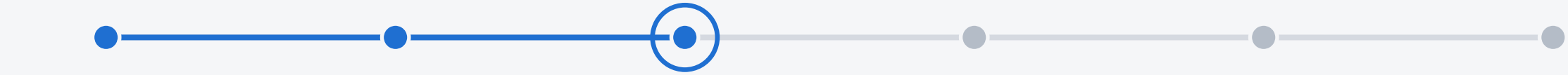
sudo add-apt-repository -y universe
```

Updates Ubuntu, installs a few tools we need (curl, git), sets the language to UTF-8, and turns on Ubuntu's universe packages.

Note

sudo asks for your Ubuntu password. Type it and press Enter, even though nothing shows.

Installing on Windows (WSL2)



Ubuntu terminal

```
export ROS_APT_SOURCE_VERSION=$(curl -s \
  https://api.github.com/repos/ros-infrastructure/ros-apt-source/releases/latest \
  | grep -F "tag_name" | awk -F'"' '{print $4}')
V=$ROS_APT_SOURCE_VERSION

curl -L -o /tmp/ros2-apt-source.deb \
  https://github.com/ros-infrastructure/ros-apt-source/releases/download/$V/ros2-apt-source_$V.jammy_all.deb

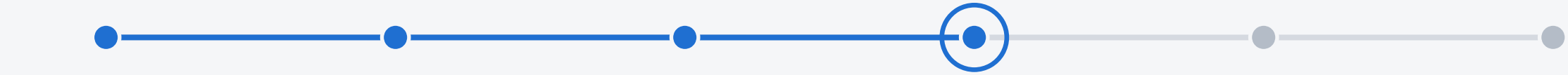
sudo dpkg -i /tmp/ros2-apt-source.deb
```

apt does not know where ROS 2 is yet. This downloads a small package that adds the official ROS 2 download source.

Note

Copy it exactly, or copy from the README. This is the method in the official ROS 2 Humble install guide.

Installing on Windows (WSL2)



Ubuntu terminal

```
sudo apt update && sudo apt upgrade -y

# ROS 2 Humble with RViz and demos, plus build tools (colcon)
sudo apt install -y ros-humble-desktop ros-dev-tools

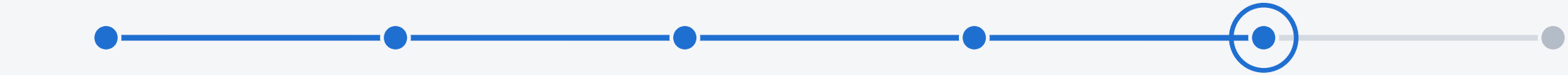
# Gazebo + its ROS plugins, robot model tools, keyboard driving
sudo apt install -y ros-humble-gazebo-ros-pkgs ros-humble-xacro \
  ros-humble-teleop-twist-keyboard
```

Installs everything the course uses: ROS 2 Humble, colcon, Gazebo 11 and its ROS plugins, xacro, and keyboard teleop.

Note

A few GB. It can take 20 to 40 minutes. Do it at home on fast Wi-Fi, before class.

Installing on Windows (WSL2)



Ubuntu terminal

```
# Load ROS 2 in every new terminal automatically
echo "source /opt/ros/humble/setup.bash" >> ~/.bashrc
source ~/.bashrc

# Test. Terminal 1:
ros2 run demo_nodes_cpp talker

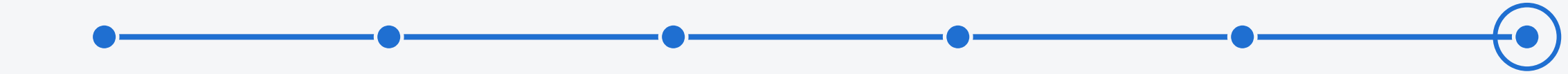
# Terminal 2 (open a second Ubuntu window):
ros2 run demo_nodes_py listener
```

~/.bashrc runs every time a terminal opens, so ROS 2 is always ready. The talker and listener are two nodes talking on a topic.

Note

Listener prints "I heard: [Hello World: 1]"? ROS 2 works. Stop both with Ctrl+C.

Installing on Windows (WSL2)



Ubuntu terminal

```
gazebo
# An empty Gazebo world opens on your Windows desktop. Close it.

# Black window or crash? Use software graphics:
echo "export LIBGL_ALWAYS_SOFTWARE=1" >> ~/.bashrc
source ~/.bashrc
```

WSL2 shows Linux windows (WSLg) right on your Windows desktop, so Gazebo opens like a normal app.

Note

First start is slow. Update your Windows graphics driver (Intel, AMD or NVIDIA) for smoother Gazebo.

Running the simulation

```
# 1. Get the code
mkdir -p ~/rosmaster_ws/src && cd ~/rosmaster_ws/src
git clone https://github.com/Phearamony/ROSMASTER_crash_course.git

# 2. Build it
cd ~/rosmaster_ws && colcon build --symlink-install
source install/setup.bash

# 3. Terminal 1: open Gazebo with the A1
ros2 launch rosmaster_sim sim.launch.py

# 4. Terminal 2 (source first!): start the brain
ros2 run rosmaster_avoidance lidar_avoidance
```

Step 1 of 4

Get the code

mkdir makes the workspace and src/. git clone downloads the course repo into src/.

src/ now has rosmaster_sim and rosmaster_avoidance.

Running the simulation

```
# 1. Get the code
mkdir -p ~/rosmaster_ws/src && cd ~/rosmaster_ws/src
git clone https://github.com/Phearamony/ROSMASTER_crash_course.git

# 2. Build it
cd ~/rosmaster_ws && colcon build --symlink-install
source install/setup.bash

# 3. Terminal 1: open Gazebo with the A1
ros2 launch rosmaster_sim sim.launch.py

# 4. Terminal 2 (source first!): start the brain
ros2 run rosmaster_avoidance lidar_avoidance
```

Step 2 of 4

Build and source

colcon build puts the built packages in install/. With --symlink-install you can edit the .py file without building again.

source install/setup.bash lets this terminal find them.

Running the simulation

```
# 1. Get the code
mkdir -p ~/rosmaster_ws/src && cd ~/rosmaster_ws/src
git clone https://github.com/Phearamony/ROSMASTER_crash_course.git

# 2. Build it
cd ~/rosmaster_ws && colcon build --symlink-install
source install/setup.bash

# 3. Terminal 1: open Gazebo with the A1
ros2 launch rosmaster_sim sim.launch.py

# 4. Terminal 2 (source first!): start the brain
ros2 run rosmaster_avoidance lidar_avoidance
```

Step 3 of 4

Open the simulation

The launch file starts Gazebo with the room, adds the A1 model with its LiDAR, and starts robot_state_publisher.

The robot doesn't move yet. It publishes /scan but nothing is sending /cmd_vel.

Running the simulation

```
# 1. Get the code
mkdir -p ~/rosmaster_ws/src && cd ~/rosmaster_ws/src
git clone https://github.com/Phearamony/ROSMASTER_crash_course.git

# 2. Build it
cd ~/rosmaster_ws && colcon build --symlink-install
source install/setup.bash

# 3. Terminal 1: open Gazebo with the A1
ros2 launch rosmaster_sim sim.launch.py

# 4. Terminal 2 (source first!): start the brain
ros2 run rosmaster_avoidance lidar_avoidance
```

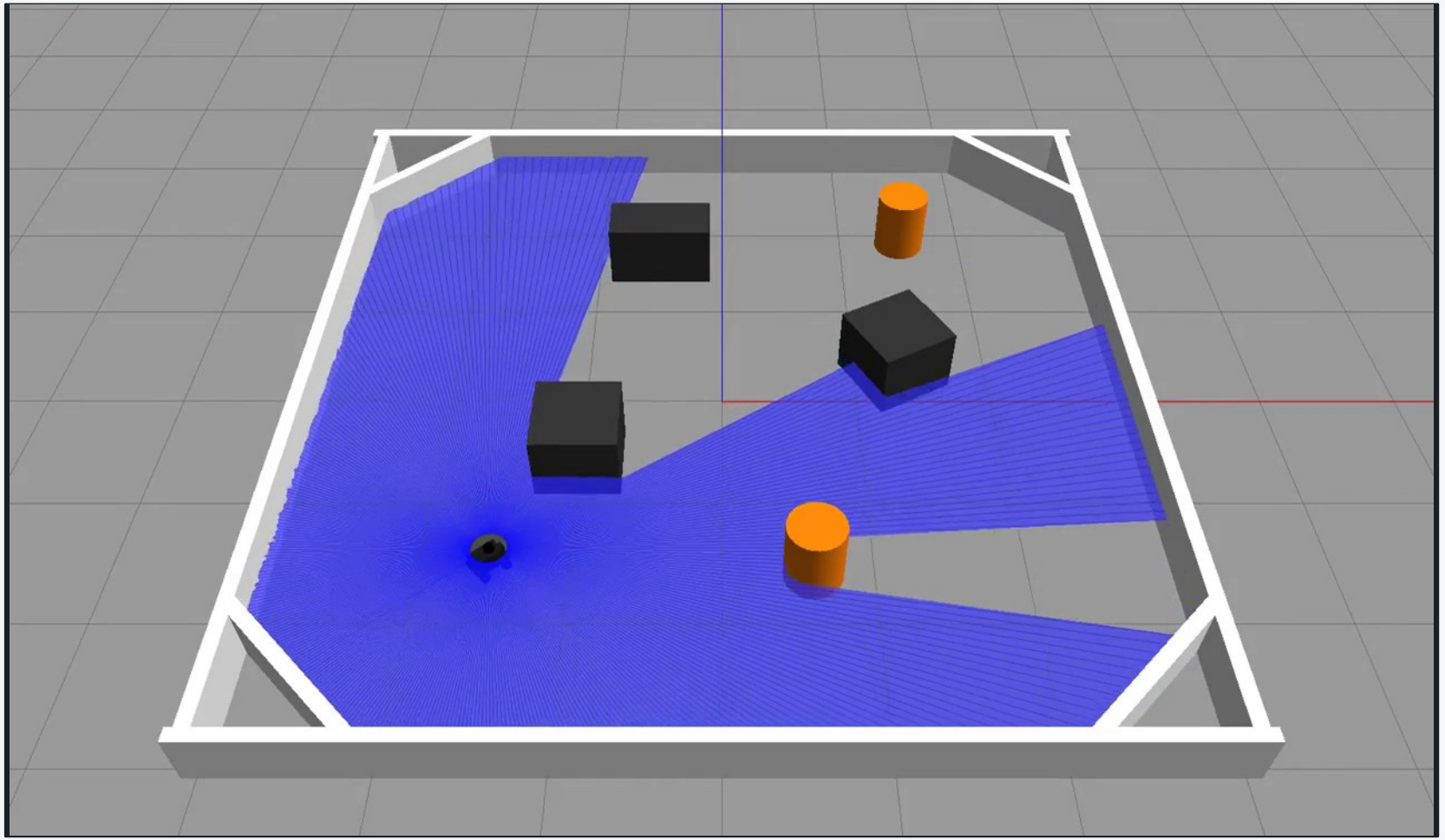
Step 4 of 4

Start the avoidance node

In a second terminal (source it first), run our node. It reads /scan and publishes /cmd_vel, so the robot starts driving.

It prints each decision, like: FORWARD front=2.31 left=1.05 right=0.88

What you should see



Recorded on a normal Windows PC with WSL2.

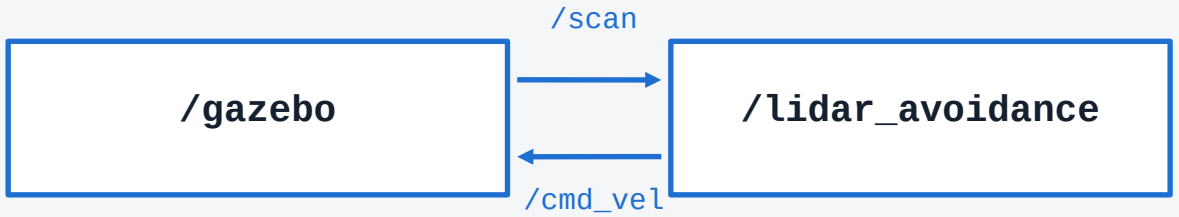
The robot drives forward, slows down near a box, and turns toward the side with more room. The terminal prints each decision.

If yours looks like this, you're done with the setup.

Checking that it works

```
# Terminal 3 (source first!)
ros2 node list             # see lidar_avoidance?
ros2 topic list            # see /scan and /cmd_vel?
ros2 topic echo /cmd_vel   # the speeds our node sends
rqt_graph                  # draw the graph
```

rqt_graph should look like this:

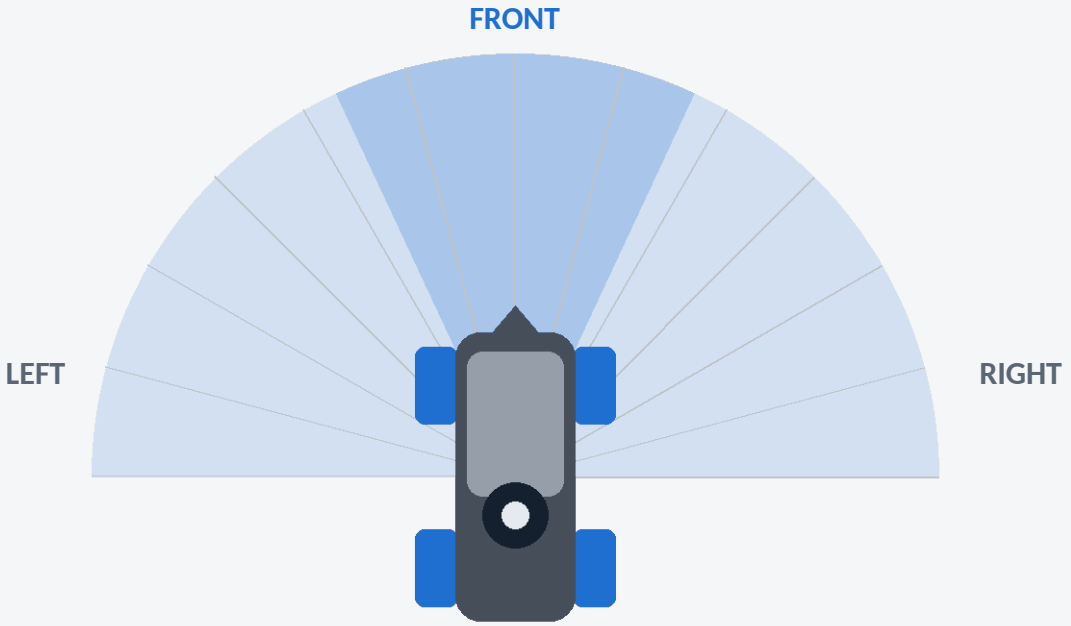


Course repo

github.com/Phearamony/ROSMASTER_crash_course

Install guide and fixes are in the README

What the LiDAR gives us



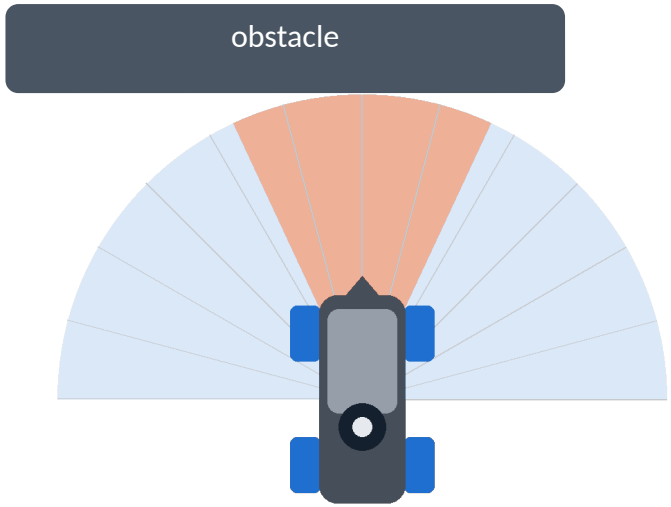
The LiDAR spins and measures the distance to the closest object in each direction, about 10 times a second. Those 360 numbers are the /scan message.

Our node only looks at three zones and keeps the smallest distance in each:

`front`	-25° to 25°
`left`	25° to 90°
`right`	-90° to -25°

0° is straight ahead. Positive angles are to the left.

The avoidance rules



1 **Front is clear**

Drive forward.

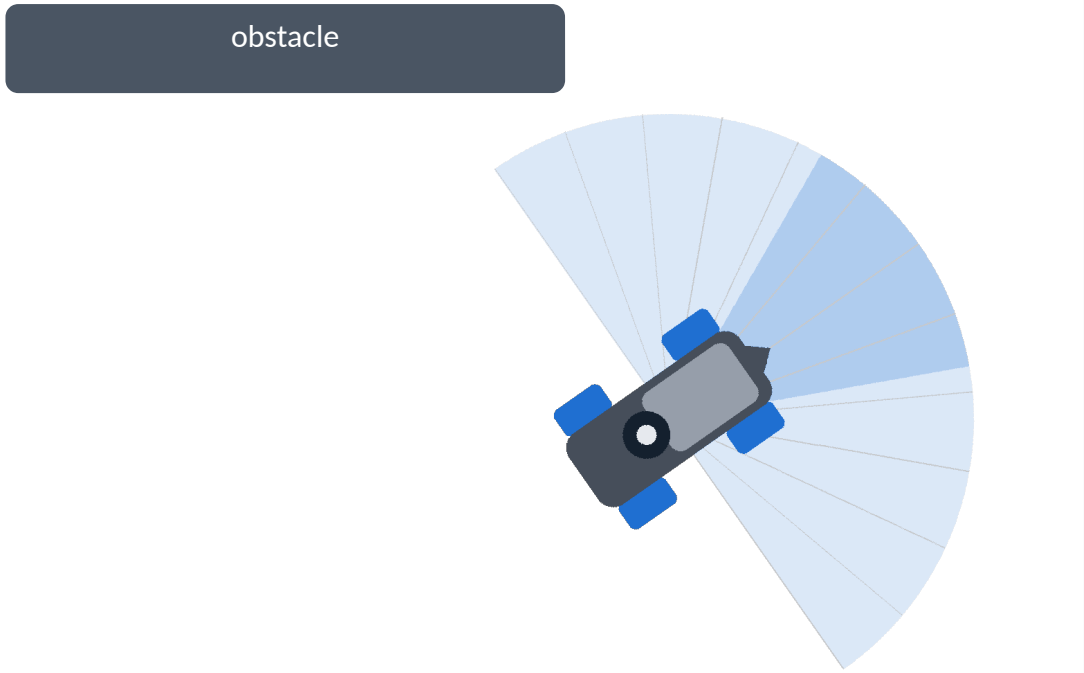
2 **Something ahead**

Slow down and steer to the side with more room.

3 **Too close**

Reverse for about 1.5 seconds, then try again.

The avoidance rules



1 **Front is clear**

Drive forward.

2 **Something ahead**

Slow down and steer to the side with more room.

3 **Too close**

Reverse for about 1.5 seconds, then try again.

Inside lidar_avoidance.py

```
class LidarAvoidance(Node):
    def __init__(self):
        super().__init__('lidar_avoidance')
        self.declare_parameter('forward_speed', 0.25)
        self.declare_parameter('safe_distance', 0.7)
        ...
        self.scan_sub = self.create_subscription(
            LaserScan, '/scan', self.scan_callback, 10)
        self.cmd_pub = self.create_publisher(
            Twist, '/cmd_vel', 10)

    def scan_callback(self, scan):
        front = self.sector_min(scan, -25, 25)
        left = self.sector_min(scan, 25, 90)
        right = self.sector_min(scan, -90, -25)

        cmd = Twist()
        if front < stop:          # rule 3: back up
            cmd.linear.x = -0.15
        elif front < safe:        # rule 2: turn
            cmd.linear.x = forward_speed * 0.6
            cmd.angular.z = turn_speed if left > right else -turn_speed
        else:                     # rule 1: go
            cmd.linear.x = forward_speed
        self.cmd_pub.publish(cmd)
```

Shortened. The full file is in the repo.

Part 1 of 4

Topics in and out

The node subscribes to /scan and makes a publisher for /cmd_vel. Each time a scan arrives, ROS 2 calls scan_callback.

There is no loop waiting for data. ROS 2 calls the function for us.

Inside lidar_avoidance.py

```
class LidarAvoidance(Node):
    def __init__(self):
        super().__init__('lidar_avoidance')
        self.declare_parameter('forward_speed', 0.25)
        self.declare_parameter('safe_distance', 0.7)
        ...
        self.scan_sub = self.create_subscription(
            LaserScan, '/scan', self.scan_callback, 10)
        self.cmd_pub = self.create_publisher(
            Twist, '/cmd_vel', 10)

    def scan_callback(self, scan):
        front = self.sector_min(scan, -25, 25)
        left = self.sector_min(scan, 25, 90)
        right = self.sector_min(scan, -90, -25)

        cmd = Twist()
        if front < stop:          # rule 3: back up
            cmd.linear.x = -0.15
        elif front < safe:        # rule 2: turn
            cmd.linear.x = forward_speed * 0.6
            cmd.angular.z = turn_speed if left > right else -turn_speed
        else:                     # rule 1: go
            cmd.linear.x = forward_speed
        self.cmd_pub.publish(cmd)
```

Part 2 of 4

Parameters

Settings with default values. You can change them from the terminal without touching the code.

```
--ros-args -p
safe_distance:=1.0
```

Inside lidar_avoidance.py

```
class LidarAvoidance(Node):
    def __init__(self):
        super().__init__('lidar_avoidance')
        self.declare_parameter('forward_speed', 0.25)
        self.declare_parameter('safe_distance', 0.7)
        ...
        self.scan_sub = self.create_subscription(
            LaserScan, '/scan', self.scan_callback, 10)
        self.cmd_pub = self.create_publisher(
            Twist, '/cmd_vel', 10)

    def scan_callback(self, scan):
        front = self.sector_min(scan, -25, 25)
        left = self.sector_min(scan, 25, 90)
        right = self.sector_min(scan, -90, -25)

        cmd = Twist()
        if front < stop:          # rule 3: back up
            cmd.linear.x = -0.15
        elif front < safe:        # rule 2: turn
            cmd.linear.x = forward_speed * 0.6
            cmd.angular.z = turn_speed if left > right else -turn_speed
        else:                     # rule 1: go
            cmd.linear.x = forward_speed
        self.cmd_pub.publish(cmd)
```

Part 3 of 4

Reading the zones

sector_min checks all 360 distances and returns the closest one inside a zone.

```
front=0.62  left=1.80
right=0.45
```

Inside lidar_avoidance.py

```
class LidarAvoidance(Node):
    def __init__(self):
        super().__init__('lidar_avoidance')
        self.declare_parameter('forward_speed', 0.25)
        self.declare_parameter('safe_distance', 0.7)
        ...
        self.scan_sub = self.create_subscription(
            LaserScan, '/scan', self.scan_callback, 10)
        self.cmd_pub = self.create_publisher(
            Twist, '/cmd_vel', 10)

    def scan_callback(self, scan):
        front = self.sector_min(scan, -25, 25)
        left = self.sector_min(scan, 25, 90)
        right = self.sector_min(scan, -90, -25)

        cmd = Twist()
        if front < stop:          # rule 3: back up
            cmd.linear.x = -0.15
        elif front < safe:        # rule 2: turn
            cmd.linear.x = forward_speed * 0.6
            cmd.angular.z = turn_speed if left > right else -turn_speed
        else:                     # rule 1: go
            cmd.linear.x = forward_speed
        self.cmd_pub.publish(cmd)
```

Part 4 of 4

The decision

The command is a Twist message. linear.x is forward speed in m/s. angular.z is turning, positive is left. Then we publish it.

In Gazebo our A1 model treats angular.z as a steering angle.

Experiments

Drive it yourself

Stop the avoidance node first.

```
ros2 run teleop_twist_keyboard teleop_twist_keyboard
```

Look at the data

Park near a box and find the small numbers.

```
ros2 topic echo /scan --once
```

Change safe_distance

Try 0.4 and 1.0. What changes?

```
ros2 run rosmaster_avoidance lidar_avoidance \
  --ros-args -p safe_distance:=1.0
```

Add obstacles

Use the box and cylinder tools in the Gazebo toolbar. Can it still get out?

Next week: reading the code

```
def scan_callback(self, scan):
    front = self.sector_min(scan, -25, 25)
    left = self.sector_min(scan, 25, 90)
    right = self.sector_min(scan, -90, -25)

    cmd = Twist()
    if front < safe:
        cmd.linear.x = forward_speed * 0.6
        cmd.angular.z = turn_speed if left > right else -turn_speed
```

Python

variables, if, functions, classes

Bash

cd, ls, nano, source, sudo apt

Linux

files, folders, permissions, the terminal

Homework: do the four experiments and bring one question.